

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 1.3

Build a Prompt That Survives Edge Cases

Module	1 — LLM Foundations & Vibe Coding with Claude
Topic	1.3 — Prompt Engineering That Holds Up in Production
Estimated time	35–40 minutes
Tool needed	Claude Console (console.anthropic.com) — no coding required
Prerequisite	Have read Module 1, Topic 1.3 on the program website first

What You're Building

GOAL

A single prompt that reliably turns a product name into a structured JSON object — `{ "title": string, "description": string, "seo_keywords": string[] }` — across four different kinds of input, not just the easy case.

This guide walks you through the exact same territory as the "Lab: build a prompt that survives edge cases" section on the Module 1 page, but step by step, with what to type, what to expect, and how to fix it when it doesn't work the first time. Work through it in order — each step builds on the last.

Steps 1–4: Build the Prompt

1 Start with the naive prompt

Open the Claude Console, start a new chat, and send exactly this — nothing more:

```
Write a product description for: Trailblazer 40L Hiking Backpack
```

What you'll see: a plain paragraph of marketing copy. No structure, no JSON — just prose. That's expected. This is your baseline, not your answer.

2 Add the output shape

Replace your prompt with a version that states the required shape explicitly:

```
Respond with only a valid JSON object, no markdown fences, no text outside the object, matching exactly this shape:
```

```
{ "title": string, "description": string, "seo_keywords": string[] }
```

```
Product: Trailblazer 40L Hiking Backpack, waterproof, 12 external pockets
```

What you'll see: a JSON object — usually correct, but check carefully. Claude sometimes wraps JSON in ````json` fences even when told not to, or adds a sentence before/after it. Note whether that happened; you'll fix it in Step 4 if so.

3 Split into system + user prompt

In the Claude Console, use the separate "System prompt" field for the durable instructions, and keep only the product info in the main message:

SYSTEM PROMPT:

"You are a product copywriter for an outdoor gear store.

Always respond with a single JSON object matching:

```
{ title: string, description: string, seo_keywords: string[] }
```

Never include text outside the JSON object. No markdown fences."

USER MESSAGE:

"Product: Trailblazer 40L Hiking Backpack, waterproof, 12 external pockets"

Why this helps: separating "how to behave" (system) from "what to do this turn" (user) is what lets you reuse this exact system prompt for every product later, changing only the user message.

4 Add one example (few-shot)

If Step 3's output still isn't 100% consistent, add one worked example directly in the system prompt, right after the schema:

Example:

Input: "Insulated Water Bottle 1L, keeps drinks cold 24h"

Output: {"title":"Insulated Water Bottle 1L","description":"Stay refreshed all day – this insulated bottle keeps drinks cold for a full 24 hours.","seo_keywords":["insulated water bottle","cold drinks","1L bottle","hydration"]}

What you'll see: markedly more consistent formatting from here on — field order, quoting style, and tone all lock into place. This is your working prompt going into edge-case testing.

Steps 5–8: Test the Four Edge Cases

Run your Step 4 prompt against each input below *without changing it first*. Only adjust the prompt if an input actually breaks it — that's the point of testing.

5 Normal, well-formed input

Product: Insulated Water Bottle 1L, keeps drinks cold 24h

Expected: clean JSON, no surprises. If this fails, something's wrong with the base prompt from Step 4 — fix that before moving on.

6 Empty or missing product title

Product:

Common failure: the model invents a plausible-sounding but fake product, or returns broken JSON.

Fix: add to your system prompt: *"If no product information is given, respond with { "title": "", "description": "", "seo_keywords": [] } — do not invent a product."* Re-run Step 6.

7 Non-English product name

Product: Mochila de Senderismo 40L Impermeable

Common failure: nothing usually breaks structurally, but check the language of the output — did it respond in English, or switch to Spanish?

Fix (if inconsistent): add: *"Always respond in English regardless of the input product name's language."* Only add this if you actually want English-only output — for a global store you might want the opposite; either way, be explicit rather than leaving it to chance.

8 HTML / special characters in the input

Product: Trailblazer Backpack "Pro" & Co.

Common failure: the HTML tags leak into the output description verbatim, or unescaped quotes inside the input break the JSON string.

Fix: add: *"Strip any HTML tags from the input before use. Escape any double quotes inside string values so the JSON stays valid."* Re-run and confirm the output no longer contains raw tags and is still parseable.

Step 9: Final Verification Pass

9 Re-run all four, back to back

Using your final prompt (after any fixes from Steps 6–8), re-run all four inputs from Steps 5–8 in a row, without editing the prompt between them. This catches fixes that accidentally broke an earlier case.

Troubleshooting

Symptom	Likely cause	Try this
Output wrapped in ```json ... ``` fences	Model defaults to markdown formatting	Add "no markdown fences" explicitly, right next to the schema, not just once at the top
Extra sentence before/after the JSON	Instruction not strict enough	Add "Respond with only the JSON object and nothing else" as its own line
Field names vary between runs	Schema shown once, not reinforced	Add the worked example from Step 4 if you haven't yet
JSON breaks on quotes/apostrophes in input	No escaping instruction	Add explicit escaping instruction from Step 8
Output is inconsistent across identical re-runs	Temperature set high	Lower temperature toward 0 in the Console's settings panel

Your Final Prompt Template

By the end of Step 9 your system prompt should look roughly like this (yours may have different edge-case fixes depending on what you found):

```
SYSTEM PROMPT:
"You are a product copywriter for an outdoor gear store.
Always respond with only a single valid JSON object, no markdown fences,
no text outside the object, matching exactly:
{ title: string, description: string, seo_keywords: string[] }

If no product information is given, respond with
{ "title": "", "description": "", "seo_keywords": [] } – do not invent a product.
Strip any HTML tags from the input before use.
Escape any double quotes inside string values so the JSON stays valid.
Always respond in English regardless of the input product name's language.

Example:
Input: "Insulated Water Bottle 1L, keeps drinks cold 24h"
Output: {"title":"Insulated Water Bottle 1L","description":"Stay refreshed all
day – this insulated bottle keeps drinks cold for a full 24 hours.",
"seo_keywords":["insulated water bottle","cold drinks","1L bottle","hydration"]}

USER MESSAGE (per product):
"Product: {product name and details}"
```

Keep this prompt somewhere you can find it — you'll reuse this exact system prompt inside the Node.js script you build in **Lab 1.5**.

Self-Check: Are You Done?

- ☐ A normal, well-formed product name returns valid JSON
- ☐ An empty/missing product title no longer invents a fake product
- ☐ A non-English product name behaves the way you decided it should (and you did it on purpose, not by accident)
- ☐ A product name with HTML/special characters returns valid, unescaped-safe JSON with no leaked tags
- ☐ You ran all four back to back on the same final prompt version, with no edits in between

All five checked? You're done with Lab 1.3. Save your final prompt template above — it carries directly into Lab 1.5.